

Intain Verify — Loan Data Verification Copilot

System design · Data model · API surface · Validation engine · AI service · Audit trail · Trade-offs

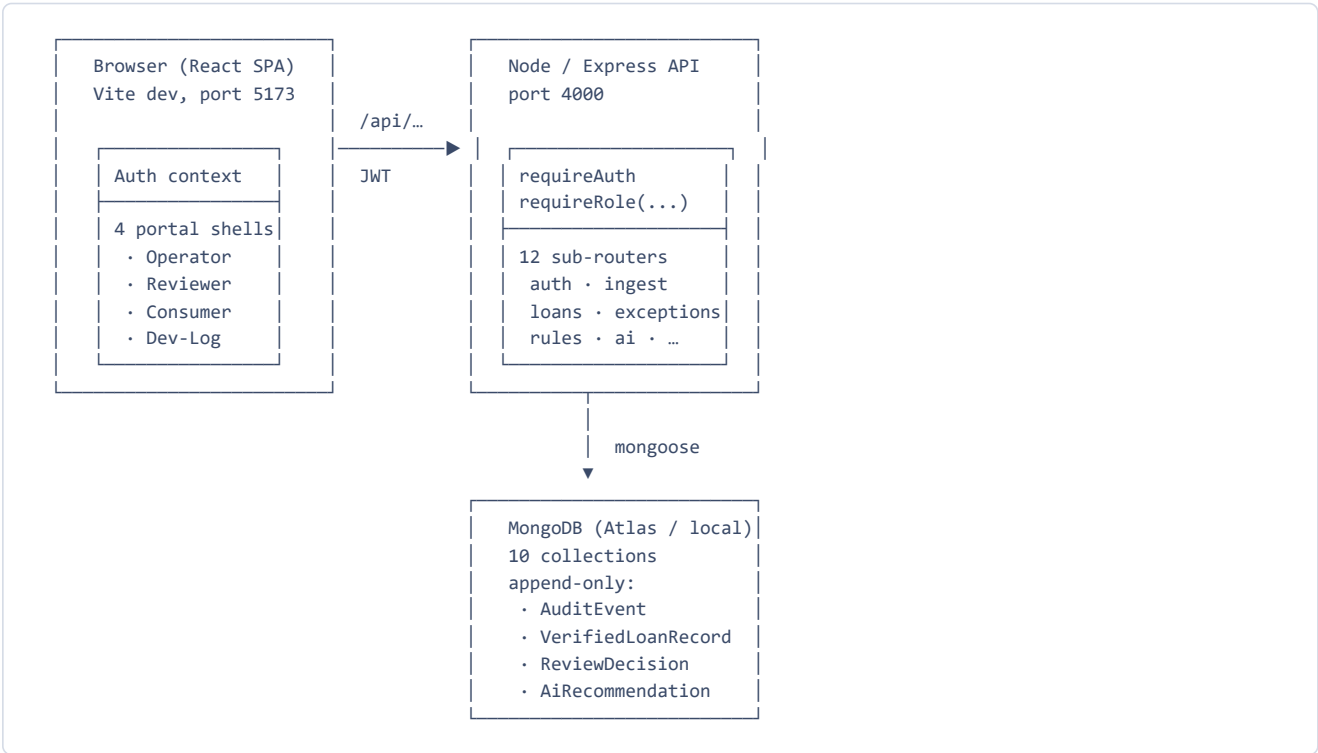
Contents

- 1. Executive summary & system design
- 2. Data model & append-only invariants
- 3. API surface & auth model
- 4. Validation engine — configuration-driven, idempotent
- 5. AI service — provider-agnostic, human-in-the-loop
- 6. Audit trail — hash-chained, tamper-evident
- 7. Trade-offs & deliberate non-goals

1 · Executive summary & system design

Intain Verify turns messy loan-tape CSVs into a verified, auditable dataset that downstream consumers can trust. Three role-based portals — Operator, Reviewer, and Consumer — sit on top of a configuration-driven validation engine, an AI review assistant, and an append-only hash-chained audit log. Data flows one direction (raw → normalized → validated → reviewed → verified → exposed read-only), and the Consumer surface is architecturally incapable of mutating anything upstream.

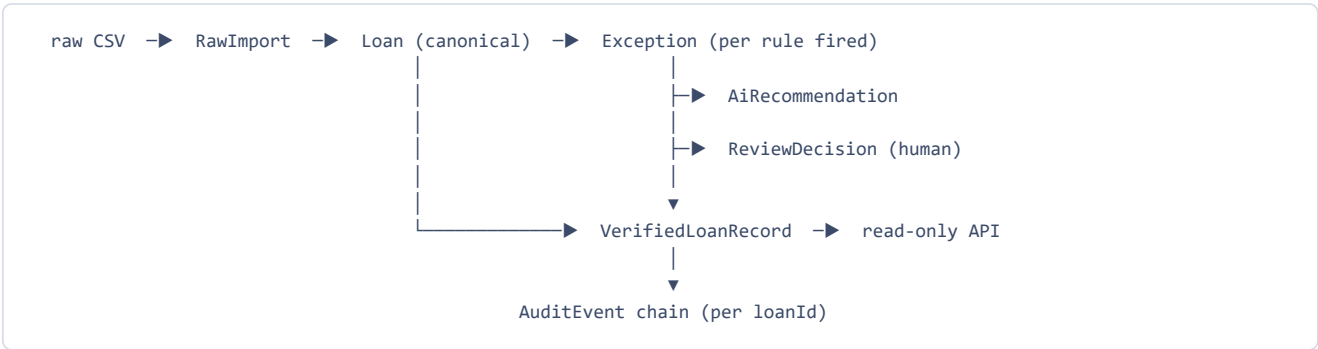
1.1 Component topology



1.2 Stack choices, with reasoning

CHOICE	REASON
React 18 + Vite	Instant dev-server boot, first-class Fast Refresh, and a build output small enough (~410 kB gzip 120 kB) to embed anywhere.
Tailwind (utility)	Zero design-system overhead, distinct portal accents applied at the layout level, no runtime CSS-in-JS cost.
Express 4	Predictable middleware chain, easy to reason about role guards on a per-route basis.
Mongoose 8	Schema-level validation catches drift before a route handler is even reached; per-collection indexes documented alongside the schema.
SHA-256 canonical hash	Node crypto built-in; deterministic ordering via a shared canonicalStringify() used both server-side and (mirrored) client-side for portable verification.
Zero AI SDK	OpenAI-compatible chat-completions endpoint is provider-agnostic — same wire format works for OpenAI, Gemini, Azure OpenAI. No vendor lock-in inside the codebase.

1.3 Data-flow, end to end

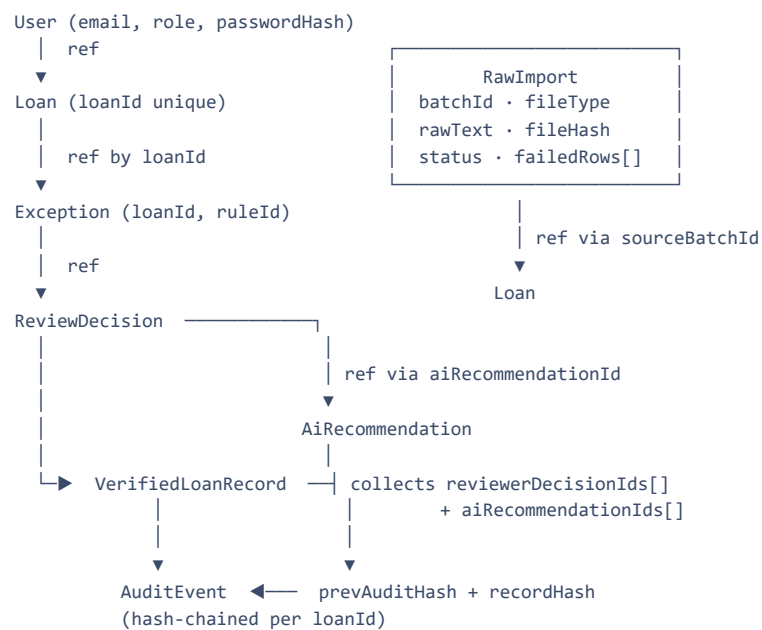


Every arrow above is **additive**. The canonical `Loan` is the only mutable document in the write path, and even its edits go through a strict whitelist. Everything downstream (exceptions, decisions, AI outputs, verified records, audit events) is append-only.

2 · Data model & append-only invariants

Ten collections, three read-only-by-design (`VerifiedLoanRecord` , `AuditEvent` , `AiRecommendation`), a hash chain that ties the audit stream to each verification, and per-collection indexes chosen to keep every list endpoint under 50 ms on the seeded dataset.

2.1 Entity map



2.2 Collections — fields, indexes, rationale

COLLECTION	KEY FIELDS	INDEXES	NOTES
User	email · role · passwordHash · name	{email:1} unique	bcrypt hash; JWT includes {sub, role, email} , 12 h expiry
RawImport	batchId · fileType · rawText · fileHash · rowCount · failedRows[]	{batchId:1} , {fileType:1}	Original CSV kept verbatim for lineage; fileHash = sha256 of raw bytes
Loan	13 canonical fields + verificationStatus , sourceBatchId , servicerUpdateBatchId	{loanId:1} unique, {borrowerId,originalPrincipal,originationDate} , {state:1} , {verificationStatus:1}	The only mutable domain document. Edits go through an 8-field whitelist.
ValidationRule	ruleId · severity · expression (JSON) · active · origin · approvedBy	{ruleId:1} , {active:1}	Only rules where active && approvedBy ever run
Exception	exceptionId · loanId · ruleId · severity · status · context · resolutionType	{loanId:1} , {status,severity,createdAt:-1} , {ruleId:1}	Upserted by {loanId,ruleId} . Never re-created if resolved/dismissed.
AiRecommendation	templateName · promptSnapshot · model · providerLatencyMs · output · confidence · fallbackUsed · createdBy	{loanId:1} , {exceptionId:1} , {templateName:1}	Append-only. Every AI call — successful or fallback — persists here.
ReviewDecision	loanId · exceptionId · action · beforeValues · afterValues · aiRecommendationId · reviewer · notes	{loanId:1} , {exceptionId:1} , {reviewer:1}	Append-only. Actions: accept_ai · edit_ai · reject_ai · manual_approve · manual_reject · request_correction · edit_field · add_comment .
VerifiedLoanRecord	loanId · snapshot · trustScore · trustBreakdown · reviewerDecisionIds[] · aiRecommendationIds[] · verifiedBy · verifiedAt · recordHash · prevAuditHash	{loanId:1,verifiedAt:-1} , {recordHash:1} unique	Append-only. Re-verification produces a new document; the "latest" read wins.
AuditEvent	loanId · type · payload · actor · actorRole · timestamp · prevHash · hash	{loanId:1,timestamp:1}	The chain. 11 event types; every state change emits one, serialized per loanId .

DevLogEntry	module · date · tool · prompt · outcome · notes · aiAuthoredPct · createdBy	{module:1} , {date:-1}	Powers the in-app development-log surface.
-------------	--	------------------------	--

2.3 Synthetic-id convention

Batch-level or otherwise-loan-less events need a stable `loanId` so the hash chain and audit filters still work. We use a documented prefix set:

<code>__BATCH__<batchId></code>	batch-level events (upload, import)
<code>__ORPHAN__servicer__<id></code>	servicer_update row with no matching Loan
<code>__ORPHAN__manifest__<id></code>	document_manifest row with no matching Loan
<code>__MISSING__<rowIdx>__<b8></code>	loan_tape row with no loan_id
<code>__RULES__</code>	rule-approval/rejection events
<code>__QUERY__</code>	natural-language Converse queries
<code>__EXCEPTION__<exId></code>	exception-scoped AI calls

3 · API surface & auth model

Fifty-eight routes across twelve sub-routers, printed as a table at server startup so the contract is discoverable without leaving the terminal. Every route falls into one of three access tiers.

3.1 Access tiers

TIER	MIDDLEWARE	APPLIES TO
Public	–	POST /api/auth/login , POST /api/auth/register , GET /api/health
Authenticated	requireAuth	All read endpoints; any role with a valid JWT
Role-gated	requireAuth + requireRole(...)	Every mutation. Precise role sets per route (operator, reviewer, admin, consumer).

3.2 Endpoint families

INGESTION

- POST /api/ingest/upload — multipart, sha256, preview
- POST /api/ingest/commit/:batchId — persist
- GET /api/ingest/imports

RULES

- GET /api/rules · POST /api/rules (admin)
- PATCH /api/rules/:ruleId/{approve,reject}
- POST /api/rules/run — re-validate

LOANS + EXCEPTIONS

- GET /api/loans , PATCH /api/loans/:loanId (whitelist)
- GET /api/loans/:loanId/reconciliation
- GET /api/exceptions , PATCH /:id/{claim,resolve} , POST /:id/comment

AI

- POST /api/ai/explain/:exceptionId
- POST /api/ai/suggest/:exceptionId
- POST /api/ai/compare/:loanId
- POST /api/ai/rule (admin)
- GET /api/ai/recommendations

VERIFIED (READ-ONLY EXPOSURE)

- GET /api/verified (alias /verified-loans)
- GET /api/verified/:loanId , /trust , /history
- GET /api/verified/export , /export/bundle
- POST /api/verified/chain-check

AUDIT + SUMMARY + CONVERSE + DEV LOG

- GET /api/audit/:loanId
- GET /api/summary{,/trust,/operator,/reviewer,/consumer}
- POST /api/converse , GET /api/converse/history
- GET /api/dev-log{,/stats} , POST , PATCH , DELETE

3.3 Design conventions

- **Read-only Consumer.** No route the Consumer role can reach ever writes to the canonical `Loan` collection. All Consumer-facing endpoints project from `VerifiedLoanRecord` .
- **Path-based aliases.** Both `/api/verified` and `/api/verified-loans` mount the same router so external tooling can use either spelling.
- **Static-path routes registered before parametric ones.** Express matches in declaration order; `/chain-check` and `/export` live above `/:loanId` in the router.
- **Deterministic filenames on downloads.** Every export sets `Content-Disposition` with a stamped filename so the download is scriptable.
- **Pagination hard-capped at 100.** Every list endpoint returns 400 for higher limits, preventing accidental full-collection scans.

4 · Validation engine — configuration-driven, idempotent

Rules are stored as JSON expressions in the `ValidationRule` collection, seeded from an external config file, and interpreted at runtime by a small pure-JS evaluator. Rule authorship is decoupled from engine code — a new rule can be added or an existing one tightened without touching a single JavaScript file.

4.1 Expression grammar

Rules are a tree of nodes. Every node has an `op` , plus operator-specific keys. Nesting composes via `and` / `or` / `not` . All operators are pure functions over the record and an `otherSources` context.

CATEGORY	OPERATORS
Field predicates	<code>isEmpty</code> · <code>notEmpty</code> · <code>equals</code> · <code>notEquals</code> · <code>lt</code> · <code>lte</code> · <code>gt</code> · <code>gte</code> · <code>inSet</code> · <code>notInSet</code> · <code>regexMatch</code>
Date predicates	<code>dateBefore</code> · <code>dateAfter</code> · <code>olderThanDays</code>
Cross-field	<code>crossFieldCompare</code> (with <code>rightMultiplier</code> support)
Cross-source	<code>existsInOtherSource</code> · <code>conflictsWithOtherSource</code> (with <code>onlyIfFresher</code> option)
Combinators	<code>and</code> · <code>or</code> · <code>not</code>

Example (from the seeded rule set):

```
{
  "ruleId": "BALANCE_EXCEEDS_PRINCIPAL",
  "severity": "high",
  "messageTemplate": "currentBalance {currentBalance} exceeds 1.05x originalPrincipal {originalPrincipal}",
  "expression": {
    "op": "crossFieldCompare",
    "left": "currentBalance",
    "op2": "gt",
    "right": "originalPrincipal",
    "rightMultiplier": 1.05
  }
}
```

4.2 Runner semantics

1. Load all rules with `active === true` && `approvedBy !== null` . AI-drafted rules that have not been human-approved never run.
2. For each loan, evaluate every rule. On match: upsert an `Exception` by `{loanId, ruleId}` .
3. **Idempotency**: if the exception already exists in status `resolved` or `dismissed` , do not re-create it — respect the human decision.
4. **Sweep**: if an existing `open` / `in_review` exception's rule no longer matches (or the rule was deactivated), auto-dismiss with a system note.
5. **Orphan handling**: rules with `appliesTo === "servicerUpdate"` or `"manifest"` iterate raw import rows, not `Loan` s, and emit under a synthetic `__ORPHAN__` id.
6. Every touched `loanId` receives one `validate` audit event; each new open exception receives an `exception_created` event.

4.3 Cross-source conflicts

The `conflictsWithOtherSource` operator matches by `loanId` against the linked servicer update row, optionally checks that the servicer update is fresher (`onlyIfFresher: true`), and reports which of the monitored fields disagree. The exception's `context` block preserves both values plus the source's `last_updated_at` , so downstream views (the reconciliation screen, the AI panel) render a side-by-side diff without re-querying.

The runner never mutates a `Loan` . It only creates, updates, or dismisses `Exception` records — preserving the "one direction of writes" invariant that keeps the data-flow diagram in section 1.3 accurate.

5 · AI service — provider-agnostic, human-in-the-loop

A single `aiService.run({templateName, input, actor, loanId, exceptionId})` entry point handles every AI use in the product. Templates encapsulate prompt engineering; the service handles provider dispatch, caching, logging, and fallback. The AI can propose, but only humans can commit.

5.1 Templates

Eight prompt templates, each with `{ system, buildUser(input), parseOutput(raw), buildFallback(input) }`. The templates enforce output shape independently of the provider:

TEMPLATE	PURPOSE
<code>explain_failure</code>	Reasoning chain for why a rule fired
<code>suggest_correction</code>	Field-level correction with confidence + source hint
<code>compare_sources</code>	Side-by-side diff summary between loan-tape and servicer-update
<code>generate_reviewer_note</code>	Templated reviewer comment for a resolution
<code>classify_severity</code>	Optional severity re-classification
<code>summarize_batch</code>	One-paragraph summary of an import batch's exceptions
<code>rule_from_nl</code>	Natural-language → ValidationRule expression tree, with dry-run count
<code>converse_query</code>	Natural-language → sanitized Mongo filter for verified data

5.2 Provider adapter (OpenAI-compatible wire format)

A thin adapter posts `{ system, user, temperature, max_tokens, response_format:{type:"json_object"} }` to ``${AI_API_BASE_URL}/chat/completions`` with a bearer token. The same wire format works for OpenAI, Google Gemini (via its OpenAI-compatibility layer), and Azure OpenAI — swap the base URL, no code change.

Configurable per environment:

<code>AI_ENABLED</code>	<code>true false</code>
<code>AI_API_KEY</code>	(bearer token)
<code>AI_API_BASE_URL</code>	<code>https://api.openai.com/v1</code> <code> https://generativelanguage.googleapis.com/v1beta/openai</code> <code> https://<resource>.openai.azure.com/openai/deployments/<dep></code>
<code>AI_MODEL</code>	<code>gpt-4o-mini gemini-2.5-flash ...</code>

5.3 Deterministic fallback

A pure-JS reasoner produces the same output shape as the provider for every template. It runs when `AI_ENABLED=false`, when the API key is empty, or when the provider errors or returns malformed JSON. Fallback outputs are marked `fallbackUsed:true` and their confidence is capped lower than a live model's would report — the UI never lies about what produced the recommendation.

The reviewer's workflow is functionally identical in both modes: same panel, same reasoning chain, same Accept / Edit / Reject controls, same audit trail. The only visible difference is a fallback pill on the model chip and a slightly lower confidence bar.

5.4 Human-in-the-loop enforcement

- Every AI call persists an `AiRecommendation` row with the full prompt snapshot, model, latency, and output before returning to the caller. If the reviewer never explicitly accepts, the record cannot have moved.

- Field mutations from `accept_ai` or `edit_ai` are gated by an eight-field whitelist (`borrowerName`, `state`, `currentBalance`, `interestRate`, `paymentStatus`, `daysPastDue`, `lastUpdatedAt`, `documentStatus`). Any other key the model returns is silently dropped and recorded in `ReviewDecision.notes.droppedFields` .
- The AI cannot reach `verificationStatus` . Verification is a distinct route with a distinct role gate.
- A 60-second in-memory cache keyed by `{templateName, exceptionId}` prevents double-click storms from producing duplicate `AiRecommendation` rows.
- An `ai_recommendation` audit event is appended for every call, so the timeline shows exactly when the AI was consulted for a given loan.

6 · Audit trail — hash-chained, tamper-evident

Every state-changing action in the product emits an `AuditEvent` , and every event for a given `loanId` is hash-linked to the previous one. Any modification to a past event breaks the chain and is detectable in $O(n)$ time.

6.1 Hash function

```
hash = sha256(canonicalStringify({
  loanId, type, payload, actor, timestamp, prevHash
}))

prevHash for event #N   = event #N-1's hash
prevHash for event #1   = "GENESIS"
```

`canonicalStringify` is a shared utility that produces byte-stable JSON regardless of key insertion order and handles `Date` / `ObjectId` / `Buffer` via their `toJSON()` methods. The identical algorithm exists client-side, which is what lets the exported bundle be re-verified in the browser.

6.2 Race protection

Concurrent requests for the same loan could otherwise interleave `appendAuditEvent` calls and corrupt the chain. The service maintains a per-`loanId` promise-based mutex — a `Map<loanId, Promise>` whose values chain so appends for the same loan serialize while different loans run in parallel.

Known limitation. The mutex is intra-process only. A horizontally scaled deployment would need a Mongo-backed lock or an equivalent leader-elect mechanism. This is documented in the service and out of scope for the single-process target.

6.3 Chain verification

`verifyChain(loanId)` loads events in chronological order and recomputes each hash from its stored fields, asserting that each event's `prevHash` matches the previous event's `hash` . It returns `{ ok, brokenAtIndex, totalEvents }` .

Three surfaces expose the result:

- `GET /api/audit/:loanId` returns the events plus `chainOk` and `brokenAtIndex` .
- `POST /api/verified/chain-check` takes a batch of loanIds and returns `{loanId: chainOk}` — used to render green-tick / red-cross chips on the verified-records list without firing N requests.
- Every verified record's audit-timeline view renders a green or red banner directly from the endpoint response.

6.4 Verified Loan Record

At verification time, the service computes:

```
recordHash = sha256(canonicalStringify({
  ...snapshot,           // all canonical loan fields at this moment
  sourceBatchId,
  servicerUpdateBatchId,
  validationSummary,
  reviewerDecisionIds,
  aiRecommendationIds,
  trustScore, trustBreakdown,
  verifiedBy, verifiedAt,
  prevAuditHash          // ties this record to the audit chain
}))
```

The record is then written append-only. Re-verification writes a new document; reads return the latest by `verifiedAt`. The uniqueness constraint is on `recordHash`, not `loanId`.

6.5 Portable integrity check

`GET /api/verified/export/bundle` emits a JSON containing the verified records, their audit trails, referenced AI recommendations, human decisions, active rules at export time, and a top-level `bundleHash`. A downstream consumer can re-hash the bundle (excluding `bundleHash`) with the same canonicalization algorithm and verify it locally, without trusting our server.

7 · Trade-offs & deliberate non-goals

Every design that ships has costs. This section documents the ones we accepted knowingly.

7.1 Correctness > throughput on audit writes

Every audit event is written by `appendAuditEvent`, one at a time, per `loanId`. Batched `insertMany` would be faster but would either force us to compute all `prevHash` values in memory before writing (fragile under concurrency) or to allow gaps in the chain (defeats the point).

ACCEPTED Per-loan serialization is $O(\text{events per loan})$, which is a small number in practice. Different loans still run in parallel.

7.2 Append-only VerifiedLoanRecord

An in-place upsert would be simpler to query. We chose append-only so re-verification produces a new hash-linked record, historical trust scores are preserved, and any "the verified record has since changed" claim is provable rather than asserted.

ACCEPTED Reads use a group-by-`loanId` aggregation to pick the latest verification; the extra pipeline stage is negligible at seed-set size.

7.3 Hash chain, not blockchain

The chain gives tamper-evidence and portable integrity without a distributed consensus protocol, wallet infrastructure, gas costs, or any of the operational surface a real chain would introduce.

ACCEPTED A single-writer chain gives detection, not prevention. Detection is exactly what the product's downstream consumer needs.

7.4 Whitelist patch instead of generic `$set`

Every field mutation from AI-derived paths goes through an eight-key allow-list. Adding a new editable field means an explicit code change in one place.

ACCEPTED This is exactly the friction we want. It's the guarantee that keeps `verificationStatus`, `loanId`, and lineage fields unreachable from AI output.

7.5 Deterministic fallback next to a live provider

Every AI template ships with a hand-written fallback that produces the same output shape. This doubles the number of code paths per template.

ACCEPTED The fallback path is what makes an on-stage AI-outage recovery invisible to the audience, and it's what keeps a fresh-clone development environment demoable without any provider account. Cost-of-writing outweighed by cost-of-not-having-it.

7.6 Config-driven validation, not compile-time

Rules live in a Mongo collection loaded at runtime, not as TypeScript types. Adding a rule doesn't restart the process. The cost is that a broken expression is only caught at evaluation time.

ACCEPTED The expression grammar is small (18 operators, all pure), and the admin dry-run surface (`POST /api/ai/rule` reports "would match N loans, sample: [...]") lets a human review a rule's impact before activation.

7.7 Deliberate non-goals

- **Real blockchain deployment** — hash chaining captures the traceability guarantee at a fraction of the operational cost.
- **OCR / document parsing** — the document manifest is a status tracker, not a document parser.
- **Underwriting or risk scoring** — this is a verification tool, not a credit model.
- **Payment or servicing workflows** — out of the stated scope.
- **Multi-tenant / SaaS bill-of-materials** — single-tenant assumption; auth is per-user, not per-organization.

7.8 What we would add next

- A distributed lock replacing the intra-process `appendAuditEvent` `mutex`, to support horizontal scaling.
- A signed `bundleHash` (server signature over the canonical bundle) so a downstream consumer can verify authenticity in addition to integrity.
- Per-rule severity classifiers driven by the `classify_severity` AI template, with an admin approval flow analogous to the rule-from-NL surface.
- A projection-based read model for the Consumer dashboard, computed at verification time, to keep aggregation constant-time as the ledger grows.